CH 3 • 3.11

Switch Statements

Clear **multi-branch selection** in Java



Compare one expression to constant case values

What is a switch statement?

A switch can clearly represent multi-branch behavior when one value is compared with constants.

expression → matching case → break

- Java evaluates the switch expression.
- The first matching case is selected.
- That case’s statements execute.
- A break normally jumps to the end of the switch.
- If no case matches, the default case executes.



Switch vs. multi-branch if-else

Both can represent the same behavior, but switch may make the programmer's intent clearer.

```
if (dogYears == 0) {
    // Print 0-14 years
}
else if (dogYears == 1) {
    // Print 15 years
}
...
else if (dogYears == 5) {
    // Print 37 years
}
else {
    // Print unknown
}
```

Many equality tests on the same variable



A switch organizes constant-value choices

Switch statement: general form

Construct 3.11.1

```
switch (expression) {
   case constantExpr1:
      // Statements
      break;

   case constantExpr2:
      // Statements
      break;

   ...

   default: // If no other case matches
      // Statements
      break;
}
```

`switch (expression)`

`case constantExpr:`

`default:`

Rules for the switch expression and cases

- The switch expression should be an integer, char, or String.
- Do not use a Boolean or floating-point type for this form of switch.
- Each case must use a constant expression, such as 2 or 'q'.
- A case expression cannot be a variable.
- With a break at the end of each case, case order does not change which constant matches.

```
Good case values
2       'q'       "yes"
```

```
Not a case constant
case myVariable:
```

Prefer a logical case order for readability.

Good practice: include a default case

The default handles values not matched by any explicit case.

```
switch (dogYears) {
    case 0:
        // known result
        break;
    case 1:
        // known result
        break;
    default:
        System.out.println("Unknown");
        break;
}
```

Why default?

It provides a path when no listed case matches.

Good practice: always include one.

What does break do?

A break ends the current case and exits the switch.



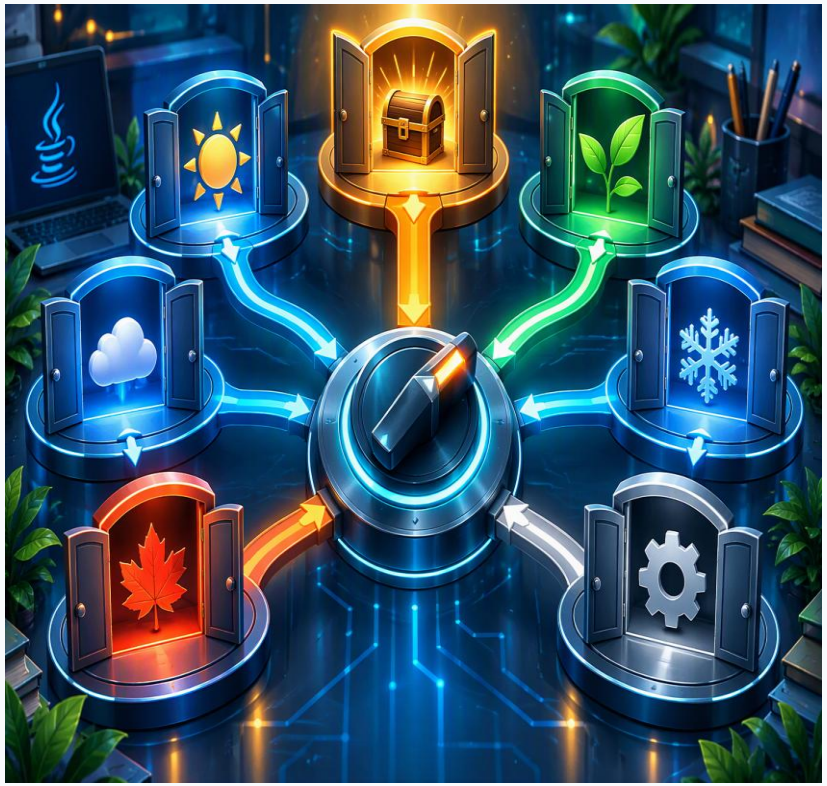
```
case 'B':
    encodedVal = 2;
    break;

// Execution continues
// after the switch
```

Common error: forgetting break may cause the next case to execute too.

Omitting break: “falling through”

Without break, execution continues into the next case.



```
switch (dogAgeMonths) {
    case 0:
    case 1:
    case 2:
        System.out.println("Same display");
        break;
}
```

Cases 0, 1, and 2 share the same statements.

Useful when several values should produce the same behavior.

Trace the switch: userChar to encodedVal

Pay close attention to where break is missing.

```
switch (userChar) {
    case 'A': encodedVal = 1; break;
    case 'B': encodedVal = 2; break;
    case 'C':
    case 'D': encodedVal = 4; break;
    case 'E': encodedVal = 5;
    case 'F': encodedVal = 6; break;
    default:  encodedVal = -1; break;
}
```

`userChar = 'C'` → `4`

`userChar = 'E'` → `6`

`userChar = 'G'` → `-1`

Why does 'C' produce 4?

```
case 'C':
case 'D':
    encodedVal = 4;
    break;
```

1. **'C' matches its case.**
2. There is no break after case 'C'.
3. Execution falls into case 'D', assigns 4, then breaks.

Result: encodedVal = 4

Why does 'E' produce 6?

```
case 'E':
    encodedVal = 5;
case 'F':
    encodedVal = 6;
    break;
```

- 'E' matches its case.
- encodedVal is first assigned 5.
- There is no break after case 'E'.
- Execution falls through to case 'F'.
- encodedVal is overwritten with 6, and then break exits the switch.

Final result: encodedVal = 6

Why does 'G' produce -1?

```
case 'A': ...
case 'B': ...
case 'C': ...
case 'D': ...
case 'E': ...
case 'F': ...
default:
   encodedVal = -1;
   break;
```

'G' matches none of A–F



default executes

Result: encodedVal = -1

3.11 Summary

Switch statements

- Use switch when one expression is compared with several constant values.
- Each case represents a possible constant match.
- Use break to stop execution and exit the switch.
- Use default for a value that matches no listed case.
- Omitting break causes fall-through into the next case.
- Fall-through can be intentional when multiple cases share the same statements.

